Campus Lost & Found: Intelligent Item Recovery Platform

CS455 Software Engineering | Deliverable 0: Project Proposal | **Group 4**

Ankit Agarwal (230151) | Rohit Yadav (230873) | Bhanu Pratap (230287) | Ankit Kumar Dubey (230154) | Dev Sachan (230353)

Students who lose items on campus currently depend on WhatsApp groups, notice boards, guards and word of mouth. This platform gives the institute one place to file lost and found reports, browse the listings with filters, and raise a verified ownership claim on any item. An AI Matching Agent returns a ranked list of similar opposite-type reports, an automated reminder loop keeps stale reports from piling up, and an SIS Guard takes custody of deposited items and settles disputed claims. Access is restricted to institute email accounts through one-time-password (OTP) login.

User Roles

Student: Signs in with an `@iitk.ac.in` address and an emailed OTP; no password exists in the system. Files lost and found reports (image, description, category, place, date/time); browses the public Lost and Found listings with filters; raises a claim request on any item; accepts or rejects claims on their own reports; participates in the item chat; edits, closes or withdraws their own reports. While filing a found report the finder may opt for *SIS Guard intervention*: they deposit the item at the SIS Guard office and their responsibility ends there — claim review and handover then belong to the guard.

SIS Guard (Admin): Privileged role for items in security custody, dispute mediation, and platform governance. Signs in through the same email-OTP flow, but on an authorized guard address — the address itself is what unlocks the guard dashboard, and again no password is stored. Sees a hidden custody queue: items awaiting deposit, items in-office, and active claims. Admins can proactively file reports for items surrendered directly to security. They contact finders to collect items, verify claimant identity in person, authorize handovers, and can manually settle, override, or close any disputed ticket. Existing Admins can provision or deactivate other Admin accounts. Every privileged action, ticket override, and account provisioning event is written to an immutable audit log.

Core Workflow

A report moves through: *created* → *listed and matched* → *claim raised* → *claim reviewed* → *handover* → *closed*, along one of two custody tracks. On the **direct track** the finder keeps the item, reviews claims and hands it over in person. On the **custody track** the finder opts for guard intervention at filing time; the report enters AWAITING DEPOSIT, the SIS Guard contacts the finder, collects the item and marks it IN CUSTODY, after which the finder is out of the loop and the guard owns claim review and handover. Bicycles always take the custody track, since a cycle cannot sit in a hostel room and is the case the security office already handles; for other categories the track is the finder's choice. Lost and found items appear on their respective public pages with the reporter's name attached. Any signed-in user may raise a claim; the claim form requires identifying details the claimant remembers but which are deliberately not published in the listing (marks, contents, lock-screen, serial, sticker). Whoever holds the item — reporter or guard — compares those details and accepts or rejects. Acceptance unlocks the handover; the report is then closed and the item cannot be claimed again. Reports may be edited or withdrawn any time before an accepted claim, and withdrawal removes the item from listings and from matching.

Automated lifecycle. A scheduled job emails the owner of every open report every three days — "have you got this item back?" — with a one-click close link, because users routinely recover items and forget to close the ticket. Reports inactive for 60 days are purged with their images by a retention job.

Shared Resource & Concurrency

The physical item is the shared, contended resource. Two students can claim the same found laptop within milliseconds, and both can be accepted by a distracted reporter in two browser tabs. Exactly one claim may reach handover. This is enforced inside a database transaction: the accept path takes a row-level lock on the item (`SELECT ... FOR UPDATE`), re-checks that the item is still in a claimable state, and a partial unique index permits at most one accepted claim per item. Losing claims are rejected with a clear message. The item state machine (OPEN → CLAIMED → RETURNED / WITHDRAWN) is the single authority on what transitions are legal.

Agentic Matching & Verification Agent

The platform integrates an autonomous orchestrator agent that replaces static search pipelines with an adaptive, multi-step reasoning loop. When filed, it extracts structured metadata from noisy text, encodes text and uploaded images into dense multimodal vector embeddings, and flags missing identifying details. It dynamically invokes tools to retrieve top-k neighbours from the vector index and applies SQL filters. If confidence is low, it autonomously broadens search parameters. On claim submission, it cross-references private verification details against hidden records and computes a match score: $s = w_1 s_{text} + w_2 s_{img} + w_3 s_{loc} + w_4 s_{time}$. It then returns a structured JSON payload containing confidence scores to the Admin/Finder dashboard.

Boundaries. The agent never accepts or rejects a claim, declares an owner, marks an item returned, posts as a user, or bypasses access control. Output is schema-validated; malformed output degrades to the deterministic filter search rather than blocking the workflow. Tool calls and candidate sets are logged.

Search & Filters

Both listing pages support filters on place, item category, date range and keyword, independently of the agent. This is the fallback path when the embedding or LLM service is unavailable, and the baseline that AI-assisted matching is measured against.

Technology Stack

Frontend: React/Next.js. Backend: Python FastAPI. Database: PostgreSQL with `pgvector`. Images: cloud object storage. Jobs: scheduled worker for the 3-day reminder and 60-day purge. Email: SMTP. AI: embedding models plus an LLM/agent framework. Auth: institute-domain email OTP with role-based access control. Dev tools: GitHub + Jira. Deployment: AWS or GCP free tier.

Design Assumptions

(i) Listings, reporter names and item chats are open to every signed-in institute user by design; the login wall keeps this inside the institute. (ii) Because listings are public, verification cannot rely on listing text: a claim's identifying details are visible only to the item holder, the claimant and the guard, and are revealed to all claimants only when a claim is contested. Publishing them would make impersonation trivial. (iii) Names are derived from the institute email and cannot be edited; hall, room and phone can be. (iv) The set of categories forced onto the custody track is configuration, not code — bicycles are the only entry for now.

Initial Risks & Uncertainties

- Fraudulent or mistaken claims, made easier by fully public listings
- Concurrent claims on one item if transactions are written incorrectly
- Choosing sensible score weights and a candidate cut-off threshold
- Agent failure modes: hallucinated matches, wrong tool choice, invalid output
- OTP abuse: spamming, brute force, mailbox delivery failures
- Items never actually deposited after opting into guard custody
- Scheduled-job faults: duplicate reminder mail, or purging a live report
- Unauthorized access to claim verification details

Mitigations: human acceptance of every claim, role-based access control, locking transactions and state constraints, OTP rate limiting, input and image validation, read-only agent permissions, idempotent jobs with a soft-delete grace window, and audit logging.

Development Strategy

Jira holds requirements, user stories, tasks and sub-tasks, bugs, priorities, estimates, sprints and the progress dashboard. GitHub holds the source, with individual accounts, feature branches, pull requests and peer review. Every significant change follows: Requirement → Jira Issue → Branch → Commit(s) → Pull Request → Test Case → Test Result.

Jira: JIRA Group 4 (CLICK HERE)

GitHub: campus-lost-and-found (CLICK HERE)